

1. Project Overview

The Smart Health Access System is a mobile application designed to close the gap between patients and healthcare providers in communities where clinic visits are difficult to arrange, medical histories are scattered across paper files, and appointment scheduling depends on long queues or phone calls that often go unanswered. The system gives patients a single place to register, search for doctors by specialty, book and track appointments, and view their own medical history, while giving doctors a structured way to manage their daily schedule and update patient records, and giving administrators the oversight tools needed to manage users, manage doctors, and generate reports on system usage.

The system directly supports United Nations Sustainable Development Goal 3 (Good Health and Well-Being), which calls for ensuring healthy lives and promoting well-being for all ages, including the target of achieving universal health coverage and access to quality essential healthcare services. By digitising appointment booking, medical record-keeping, and health reminders, the Smart Health Access System reduces missed appointments, shortens the time between a patient recognising a symptom and seeing a doctor, and gives both patients and clinicians a shared, accurate record of care over time.

1.1 Problem Statement

In many under-resourced healthcare settings, patients have no reliable way to know which doctors are available, when they can be seen, or what was discussed at their last visit. Appointments are frequently booked by walking into a clinic or by phone, which disadvantages patients who cannot easily travel or who call outside office hours. Medical records are often kept on paper, which makes it difficult for a doctor to review a patient's history quickly, increases the risk of lost information, and prevents patients from having any independent copy of their own health data.

1.2 Proposed Solution

The Smart Health Access System addresses this problem with a single mobile application serving three distinct user roles. Patients can register, search for doctors by specialty, book appointments with a clear view of availability, receive automated reminders for appointments and medication, and view or download their own medical history. Doctors can view and manage their daily appointment schedule, update patient records after a consultation, and review a patient's full history before treating them. Administrators oversee the platform itself: managing user and doctor accounts and generating reports that give a picture of overall system usage and clinic load.

1.3 Target Users

User Role	Description	Primary Needs
Patient	Member of the public seeking healthcare services	Book appointments, view records, get reminders
Doctor	Licensed medical practitioner using the platform	Manage schedule, update records, review patient history
Administrator	Platform/clinic staff managing the system	Manage users and doctors, generate usage reports

Table 1: Target user roles for the Smart Health Access System

2. Software Development Life Cycle (SDLC)

The Smart Health Access System was developed using the Iterative and Incremental SDLC model. This model was chosen over a strict Waterfall approach because the project's UI/UX design, prototype, and documentation all needed to evolve together: early feedback on the use case diagram shaped the screens that were designed in Figma, and early prototype screens in turn revealed gaps in the original requirements that had to be revisited. An iterative model allows each phase to be revisited in light of what was learned in later phases, rather than treating requirements as fixed from day one, which better reflects how the project actually progressed across the seven-week build.

Rather than completing requirements, design, implementation, and testing once in a single long pass, the project was broken into short increments, each of which produced a working, demonstrable piece of the system: first the core navigation and authentication screens, then the patient-facing booking and records features, then the doctor and admin modules, with each increment reviewed and refined before the next began. This made it possible to catch a misunderstanding about, for example, what a doctor's dashboard needed to show, well before the entire prototype was finished.

2.1 Phases of the Iterative & Incremental Model Applied to This System

Phase 1: Planning & Requirements Gathering

This phase defined the problem the system would solve, identified the three user roles (Patient, Doctor, Administrator), and translated the project brief's requirements into a concrete feature list: registration and login, doctor search, appointment booking, medical records, notifications, and the three admin functions of managing users, managing doctors, and generating reports. The SDG alignment with Goal 3 was confirmed at this stage, which anchored every later design decision to a real-world health access problem rather than a generic CRUD application.

Phase 2: System & Architecture Design

With requirements confirmed, the three-tier system architecture (presentation, application, and data layers) was designed, along with the use case diagram showing how each actor interacts with the system, the class diagram modelling the core entities (User, Patient, Doctor, Admin, Appointment, MedicalRecord, Notification), and a sequence diagram detailing the step-by-step flow of the most critical use case, booking an appointment. These diagrams were treated as living documents: the class diagram was revised once the prototype screens made clear that appointments needed a status field, for example.

Phase 3: Incremental Implementation (Prototype Build)

Implementation proceeded in increments rather than as one pass. The first increment delivered the splash screen, login, and registration flow. The second delivered the patient home dashboard, doctor search, and appointment booking screens. The third delivered medical records and notifications. The fourth delivered the doctor dashboard and admin dashboard. Each increment was reviewed against the use case diagram before the next began, which is the defining characteristic of an incremental build: working software at the end of every short cycle, not just at the very end of the project.

Phase 4: Testing & Review

Each increment was checked for internal consistency. For example, once the Book Appointment screen was built, it was checked against the Doctor Dashboard to confirm that a booking made by a patient would appear correctly in the doctor's daily appointment list, and the medical record screen was checked against the sequence diagram to confirm the data fields lined up. Because testing happened after every increment rather than only at the end, design mistakes were caught while they were still cheap to fix.

Phase 5: Deployment & Documentation

The final phase consolidated all increments into the complete prototype, finalised the GitHub repository structure, applied the MIT License, and produced this report. Although the system described here is a prototype rather than a live deployment, this phase mirrors

what a real deployment phase would involve: packaging the finished increments, documenting how the system works, and preparing it for others to use, extend, or contribute to.

2.2 Why Iterative & Incremental Fits This Project

A Digital Public Good is, by definition, meant to be reviewed, reused, and improved by others, which is much easier when the system was built in clearly separated, working increments rather than as one monolithic effort. The iterative model also matched the realistic constraint of a seven-week timeline shared across research, architecture, design, prototyping, and documentation: it allowed design and documentation work to begin on early increments while later features were still being built, instead of forcing every requirement to be finalised before any screen could be designed.

3. System Architecture Design

The Smart Health Access System follows a three-tier architecture, separating the mobile client, the application logic, and the data storage into distinct layers. This separation keeps the mobile application lightweight and focused purely on presentation, while all business rules, such as preventing two patients from booking the same doctor at the same time, live in a single, testable application layer rather than being duplicated across client code.

3.1 Architecture Diagram

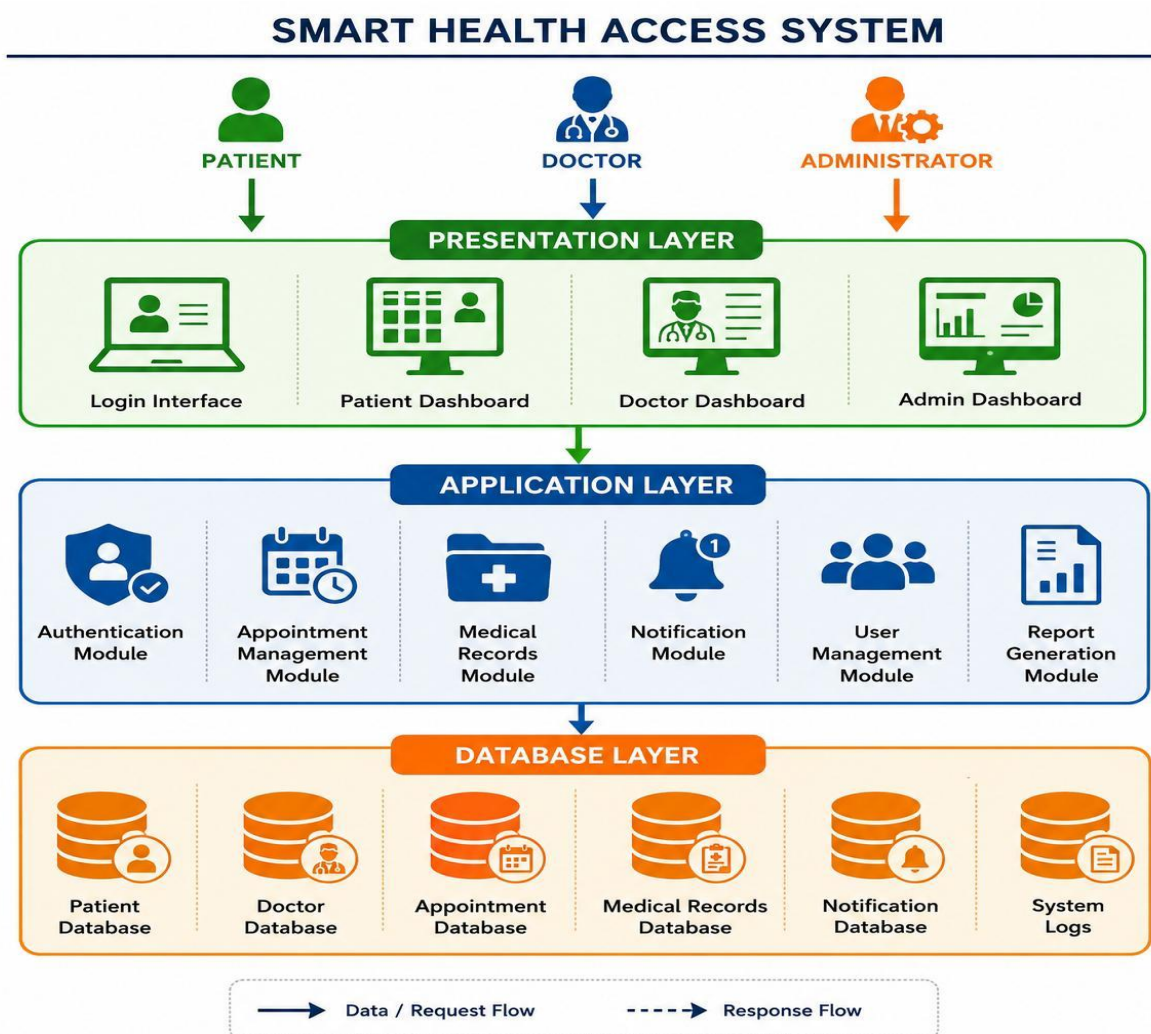


Figure 1: Three-tier system architecture of the Smart Health Access System

3.2 Layer Descriptions

Presentation Layer (Mobile Client)

Built with React Native, this layer renders the screens shown in Section 5 and is organised into three role-based modules: the Patient App Module (registration, login, booking, records, notifications, profile), the Doctor App Module (appointment list, schedule updates, patient history), and the Admin App Module (user management, doctor management, reports). A shared component library provides common UI elements, such as buttons, form inputs, and the bottom navigation bar, so the three modules stay visually consistent.

3.3 Component Interaction

When a patient books an appointment, the mobile client sends a request to the API gateway, which passes it through the validation and middleware layer for authentication and input checks before the business logic service checks for a scheduling conflict and writes the new appointment to MongoDB; once the database confirms the write, the API returns a success response that the client uses to display a booking confirmation. This same pattern, client to API gateway to business logic to database and back, applies to every feature in the system, which keeps the architecture predictable even as new features are added. Section 4.3 walks through this exact flow as a sequence diagram.

4. UML Diagrams

Three complementary UML diagrams were developed to model the Smart Health Access System from different angles: a use case diagram capturing what each actor can do, a class diagram capturing the system's underlying data structures and how they relate, and a sequence diagram capturing the step-by-step interaction behind the system's most important function, booking an appointment.

4.1 Use Case Diagram

The use case diagram identifies three actors: Patient, Doctor, and Administrator, and the functions each can perform. A patient can register or log in, book an appointment, view medical records, receive notifications, and manage their profile. A doctor can view and schedule appointments, update medical records, and view a patient's history. An administrator can manage users, manage doctors, and generate reports. Separating these use cases by actor early in the project made it straightforward to later organise the prototype itself into three matching app modules, shown in Section 5.

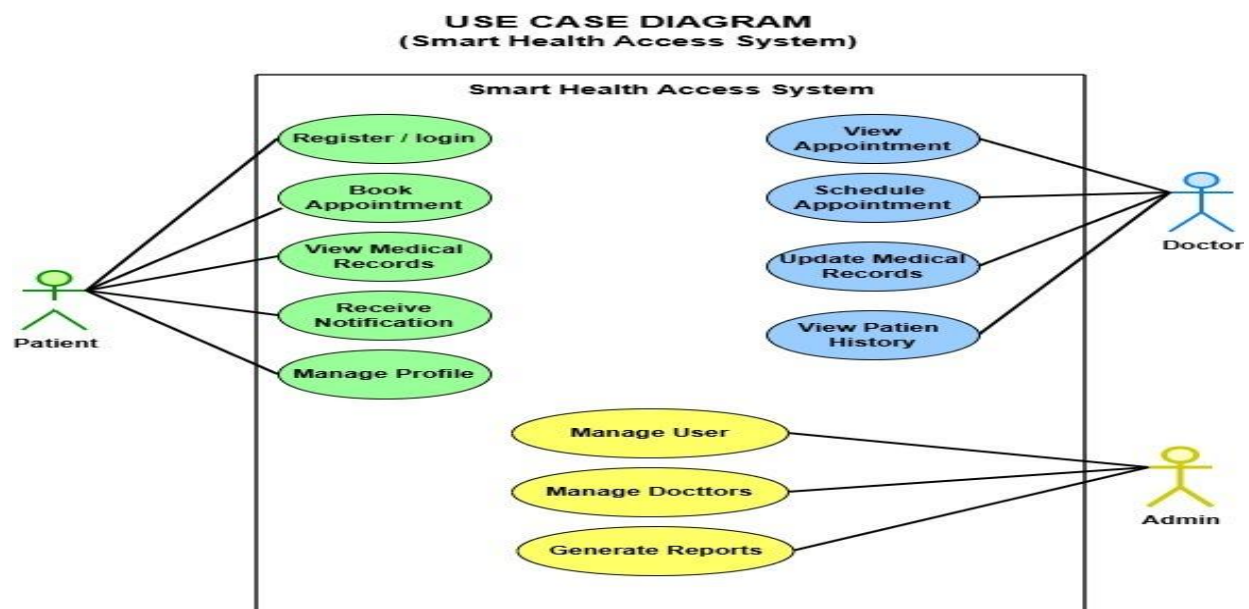


Figure 2 (reproduced from the Design of a Digital Public Good System assignment): Use Case Diagram of the Smart Health Access System

4. Sequence Diagram — Book Appointment

The sequence diagram below traces the Book Appointment use case across all three architectural layers from Section 3, showing precisely how a single patient action becomes a database write and a confirmation message. The patient opens the booking screen, the mobile app requests the current doctor list from the API, and the API queries MongoDB and returns the list for display. Once the patient selects a doctor, date, and time and confirms, the app sends a POST request to the API, which checks for a scheduling conflict before inserting the new appointment record; MongoDB acknowledges the write, and the API returns a success response that the app displays to the patient as a booking confirmation.

UML Sequence Diagram: Book Appointment Flow - Smart Health Access System

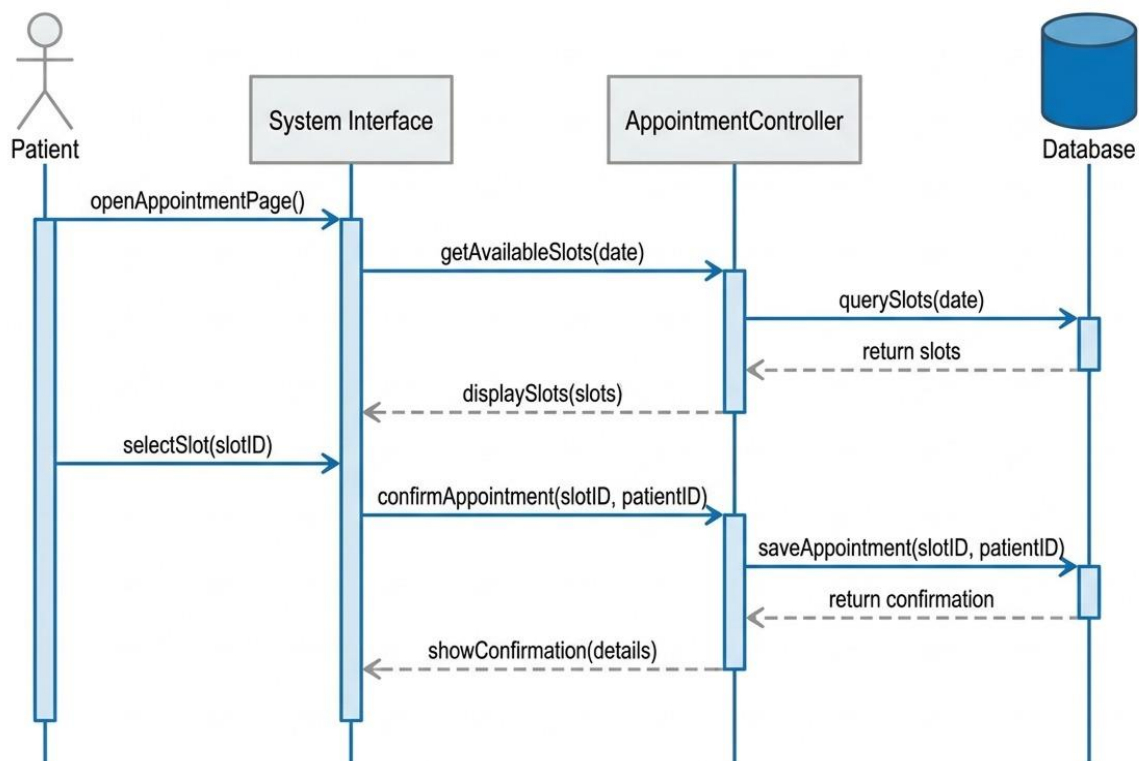


Figure 3: Sequence Diagram for the Book Appointment use case

5. Prototype Development (UI/UX Design)

The prototype was designed in Figma as a high-fidelity, role-aware mobile experience covering all three user types identified in the use case diagram. Ten key screens were designed, exceeding the brief's minimum requirement, in order to give a complete walkthrough of the patient journey from first launch through booking and records, alongside the doctor and admin views that support it. A consistent visual language, a green-and-white palette evoking health and trust, rounded cards, and a fixed bottom navigation bar for patients, was applied across every screen so the application feels like one coherent product rather than a set of disconnected mockups.

5.1 Design Principles Applied

- Consistency: the same header style, button shape, and colour palette recur on every screen so users always know where they are.
- Clarity: each dashboard (Patient, Doctor, Admin) leads with the numbers and actions that role needs most, rather than a generic menu.
- Accessibility: high-contrast text on white cards, large tappable buttons, and icons paired with text labels rather than icons alone.
- Role separation: patients, doctors, and admins each get a dashboard tailored to their use cases instead of one shared screen with hidden permissions.

Screen 1: Welcome / Splash Screen

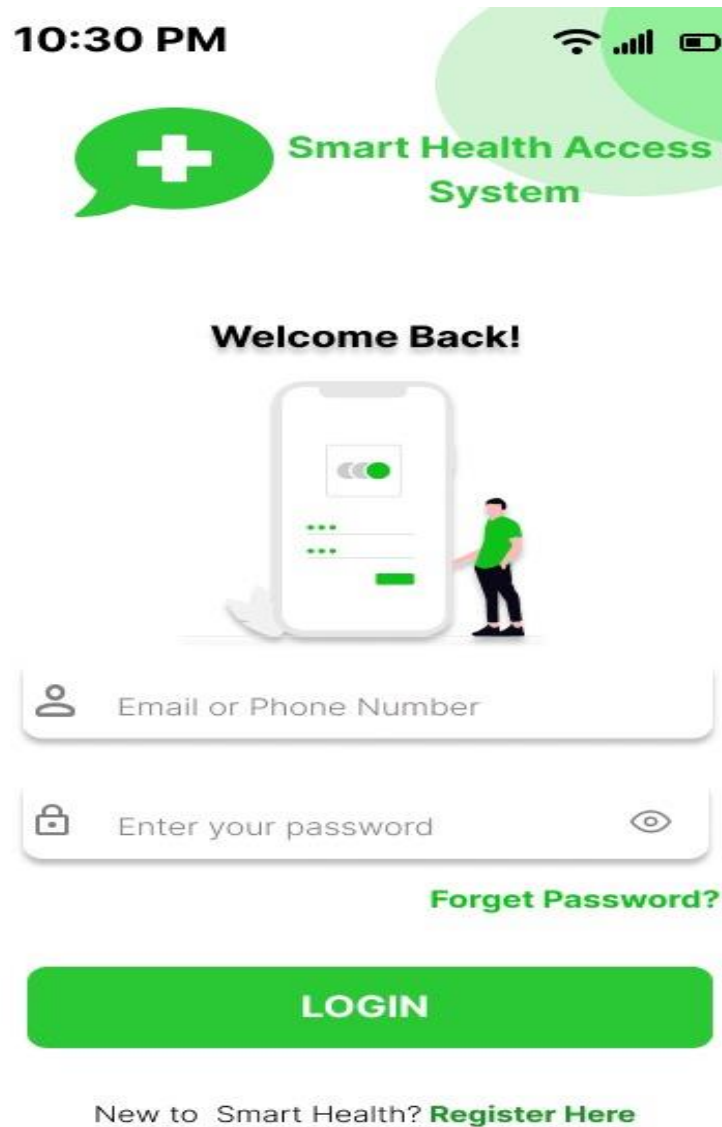
The first screen a new user sees introduces the system's purpose, "Healthcare Anytime, Anywhere," with supporting illustration and a single Get Started call to action, deliberately limiting the first impression to one decision rather than a wall of options.



Figure 4: Welcome / Splash Screen

Screen 2: Login Screen

Returning users sign in with an email or phone number and password, with a visible password-reveal toggle and a Forgot Password link. A clear Register Here link at the bottom routes new users to account creation without forcing them to hunt for it.



The login screen features a status bar at the top with the time 10:30 PM, signal strength, and battery level. Below the status bar is a green speech bubble icon with a white plus sign, followed by the text "Smart Health Access System". The main heading "Welcome Back!" is centered. Below the heading is an illustration of a smartphone with a login form on its screen, and a small figure of a person standing next to it. The login form consists of two input fields: "Email or Phone Number" and "Enter your password". The password field has a visible password-reveal toggle (an eye icon). Below the password field is a link "Forgot Password?". At the bottom is a large green button labeled "LOGIN". Below the button is the text "New to Smart Health? Register Here".

10:30 PM

Smart Health Access System

Welcome Back!

Email or Phone Number

Enter your password

Forgot Password?

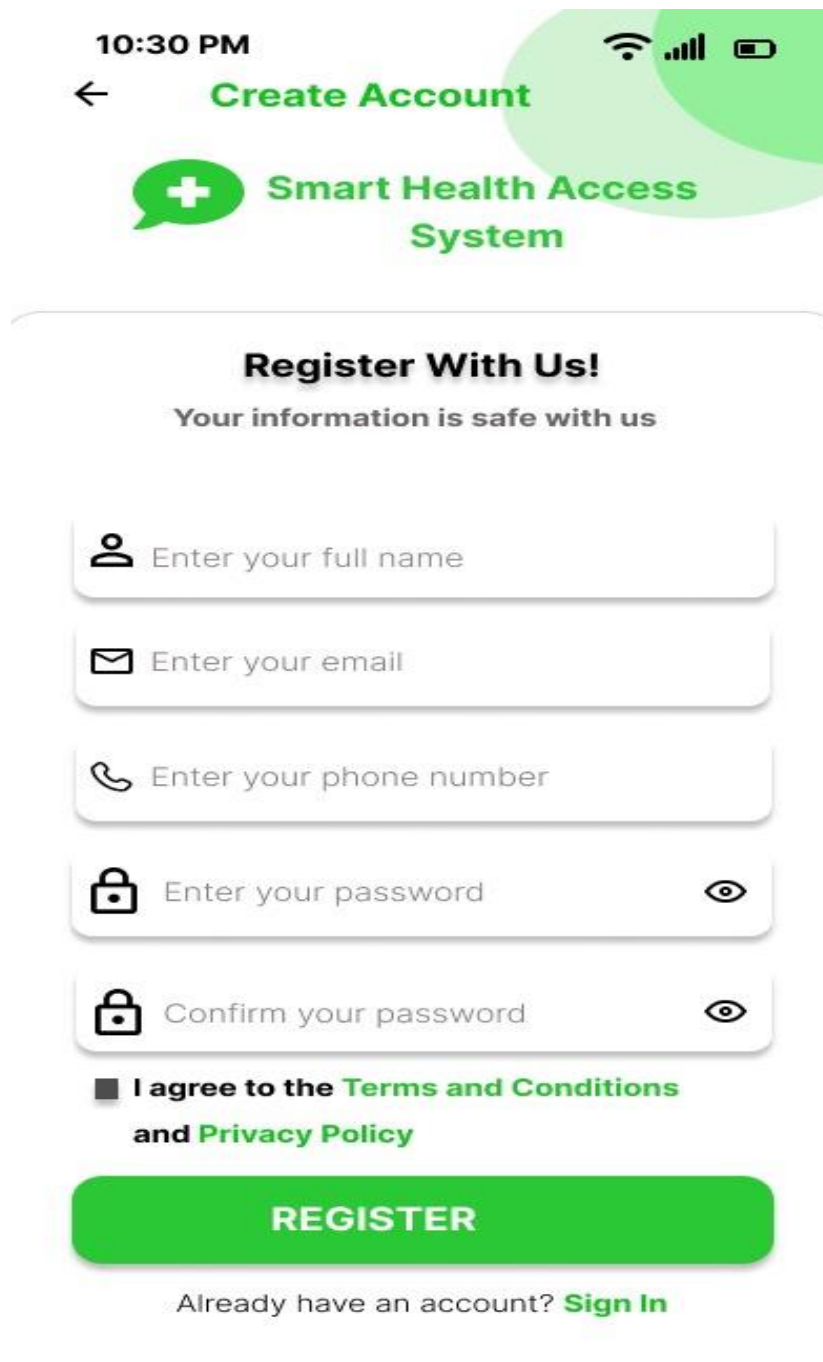
LOGIN

New to Smart Health? Register Here

Figure 5: Login Screen

Screen 3: Create Account Screen

New users provide their full name, email, phone number, and password, with password confirmation and an explicit Terms and Conditions / Privacy Policy checkbox to support informed consent, which matters in a health-data context.



The screenshot shows a mobile application interface for creating an account. At the top, the status bar displays '10:30 PM' and icons for Wi-Fi, cellular signal, and battery. Below the status bar is a navigation bar with a back arrow, the title 'Create Account', and a green circular logo with a white plus sign and the text 'Smart Health Access System'. The main content area is titled 'Register With Us!' with the subtitle 'Your information is safe with us'. It contains five input fields: 'Enter your full name' (with a person icon), 'Enter your email' (with an envelope icon), 'Enter your phone number' (with a phone icon), 'Enter your password' (with a lock icon and an eye icon for toggling visibility), and 'Confirm your password' (with a lock icon and an eye icon for toggling visibility). Below the input fields is a checkbox labeled 'I agree to the Terms and Conditions and Privacy Policy'. At the bottom is a large green button labeled 'REGISTER'. Below the button is a link that says 'Already have an account? Sign In'.

Figure 6: Create Account Screen

5.3 Patient Experience

Screen 8: Patient Home Dashboard

The patient's home screen greets them by name and surfaces the two numbers that matter most at a glance, upcoming appointments and new notifications, above a Quick Actions row giving one-tap access to booking, records, notifications, and profile.

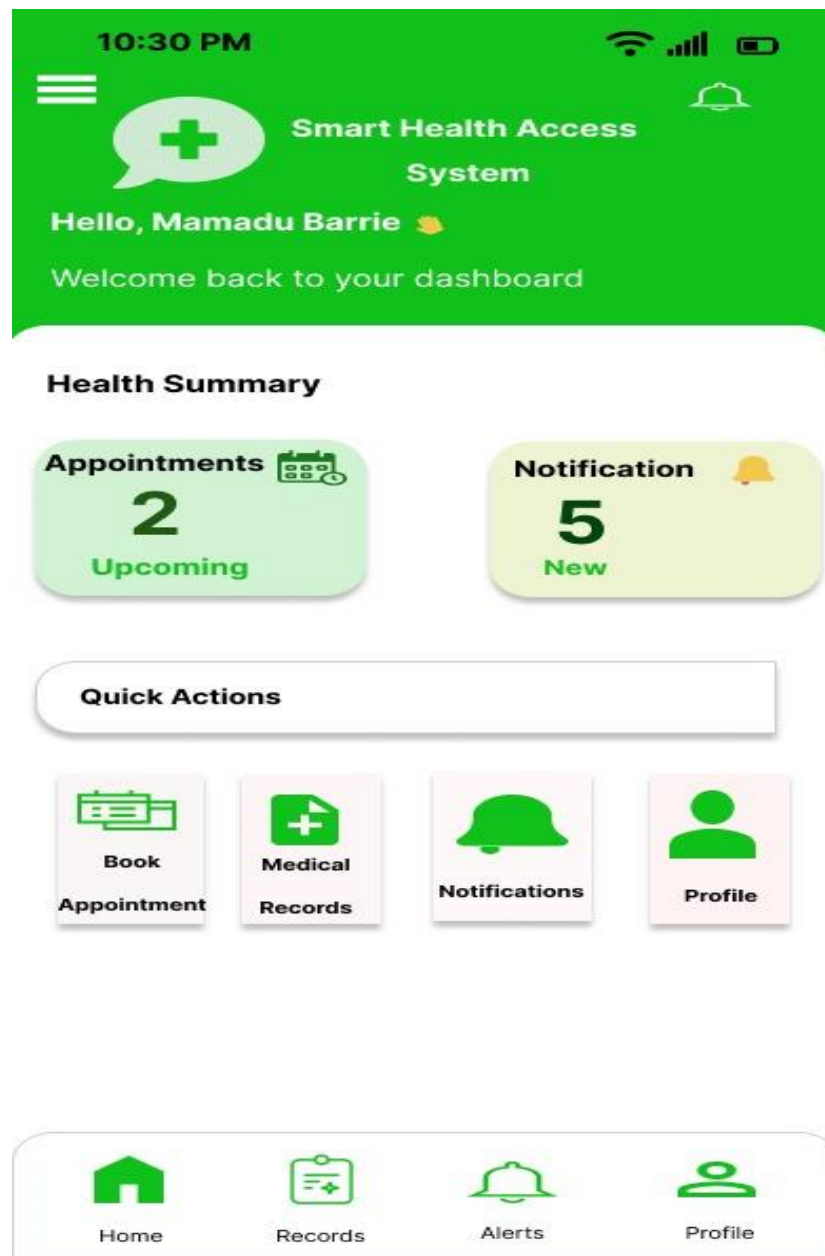


Figure 7: Patient Home Dashboard

Screen 9: Find a Doctor Screen

Patients search by doctor name or specialty and see each doctor's specialty, star rating, review count, and live availability status before tapping View Profile, so the choice of doctor is informed rather than blind.

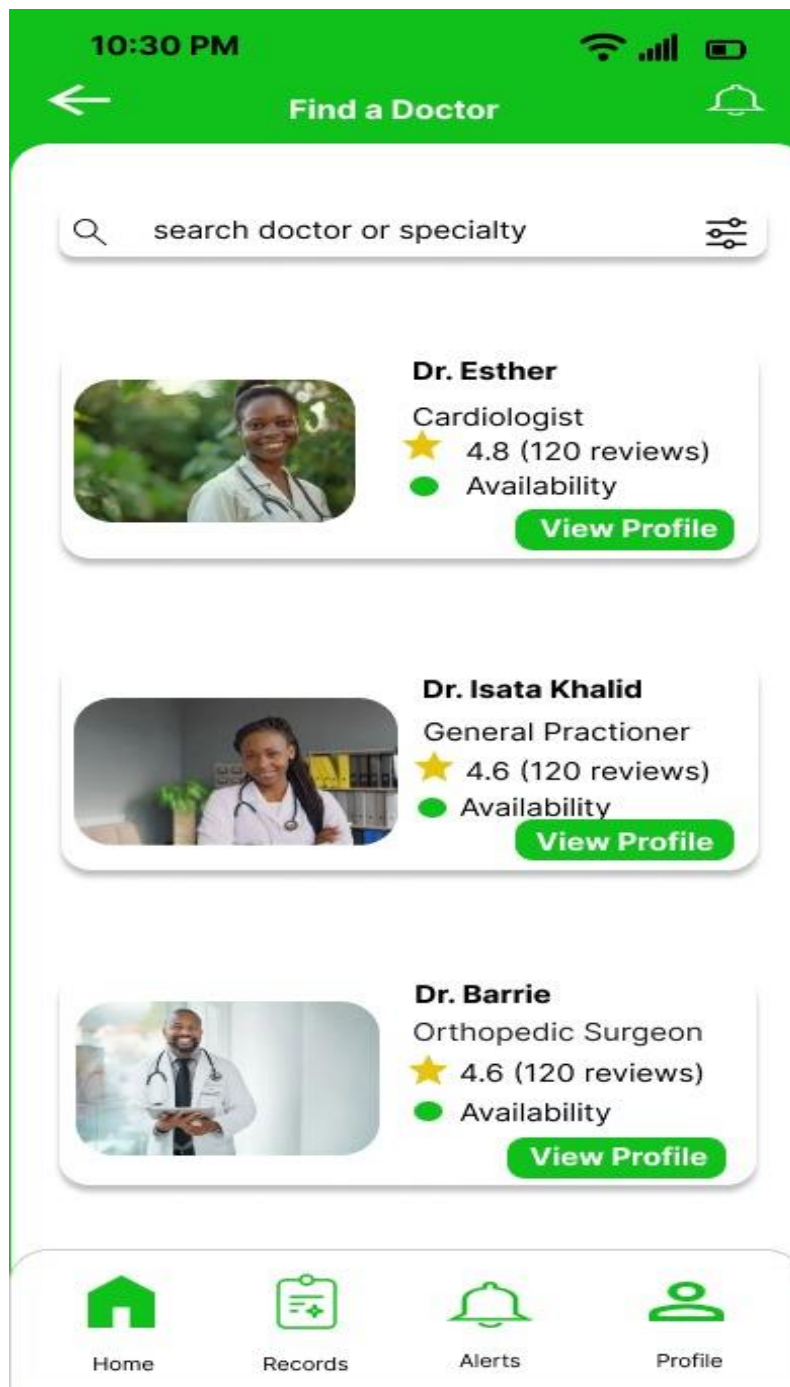


Figure 8: Find a Doctor Screen

Screen 10: Book Appointment Screen

Once a doctor is chosen, the patient selects a date and time and enters a free-text reason for the visit before confirming, giving the doctor useful context (as shown by the "Deep aching in my hips, knees, and shoulder" example) before the consultation even begins.

The screenshot shows a mobile application interface for booking an appointment. At the top, a green header bar contains the time '10:30 PM' and status icons for Wi-Fi, cellular signal, and battery. Below the header, the title 'Book Appointment' is centered. The form consists of four sections: 'Select Doctor' with a dropdown menu showing 'Dr. Barrie (Orthopedic Surgeon)'; 'Select Date' with a date picker showing '4 June 2026'; 'Select Time' with a dropdown menu showing '10:30 AM'; and 'Reason for Visit' with a text input field containing 'Deep aching in my hips, knees, and shoulder that makes movement difficult.....'. At the bottom, there is a large green button labeled 'CONFIRM BOOKING'.

Figure 9: Book Appointment Screen

Screen 11: Medical Records Screen

Patients view their own history organised into History, Prescriptions, Lab Results, and Reports tabs, with a Download Records (PDF) action that lets a patient carry a portable copy of their data to a different clinic if needed, a key digital-public-good principle of patient data ownership.

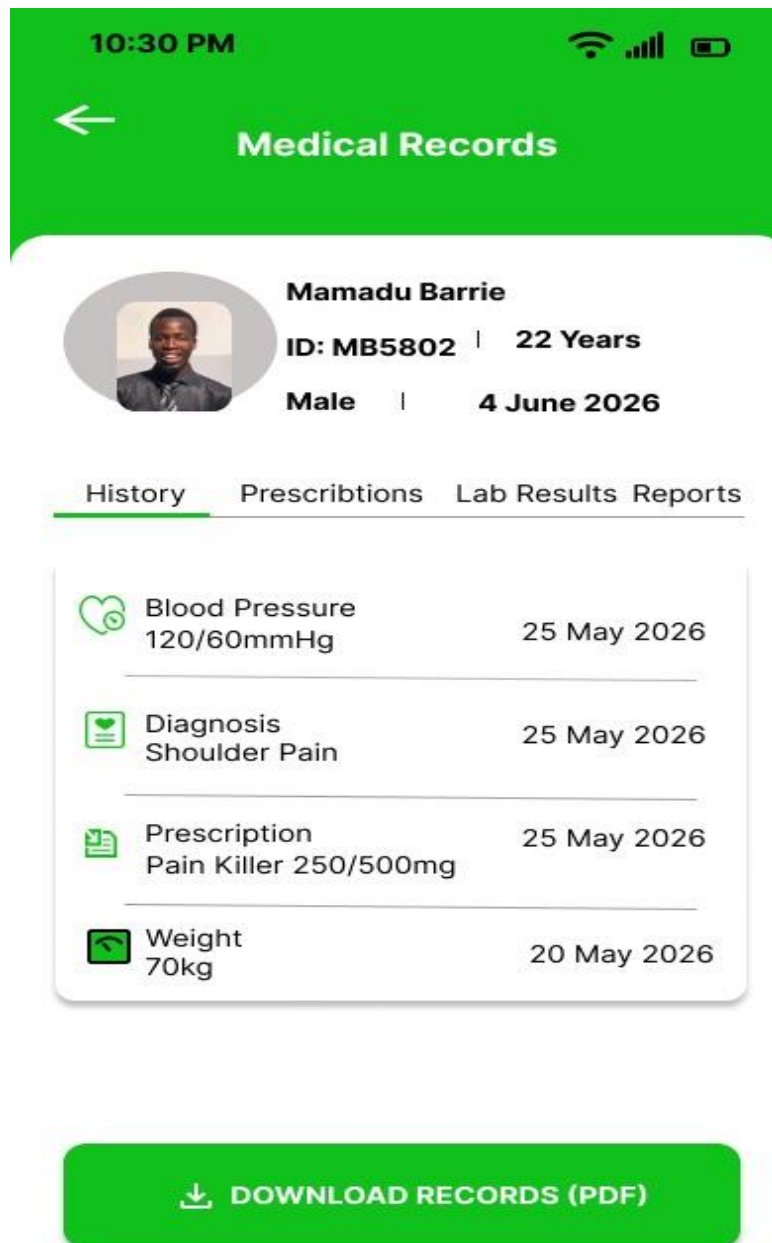


Figure 10: Medical Records Screen

Screen 12: Notifications Screen

Reminders are grouped under All, Appointments, Reminders, and Alerts filters, covering appointment reminders, medication reminders, and broader health updates such as a nearby health camp, which keeps patients engaged with their care between visits rather than only at appointment time.

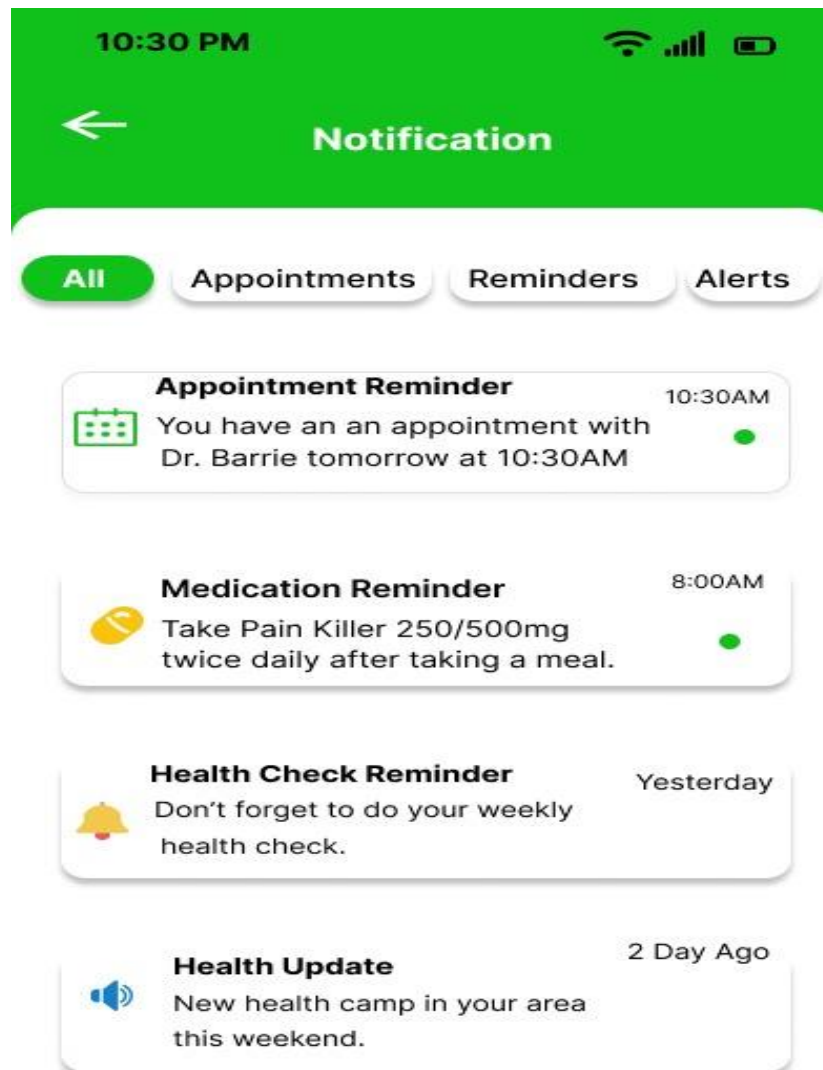


Figure 11: Notifications Screen

5.4 Doctor Experience

Screen 13: Doctor Dashboard

On logging in, a doctor immediately sees today's appointment count, how many patients are waiting, and how many consultations are already completed, followed by a same-day appointment list and direct actions to view the full schedule or update patient records.

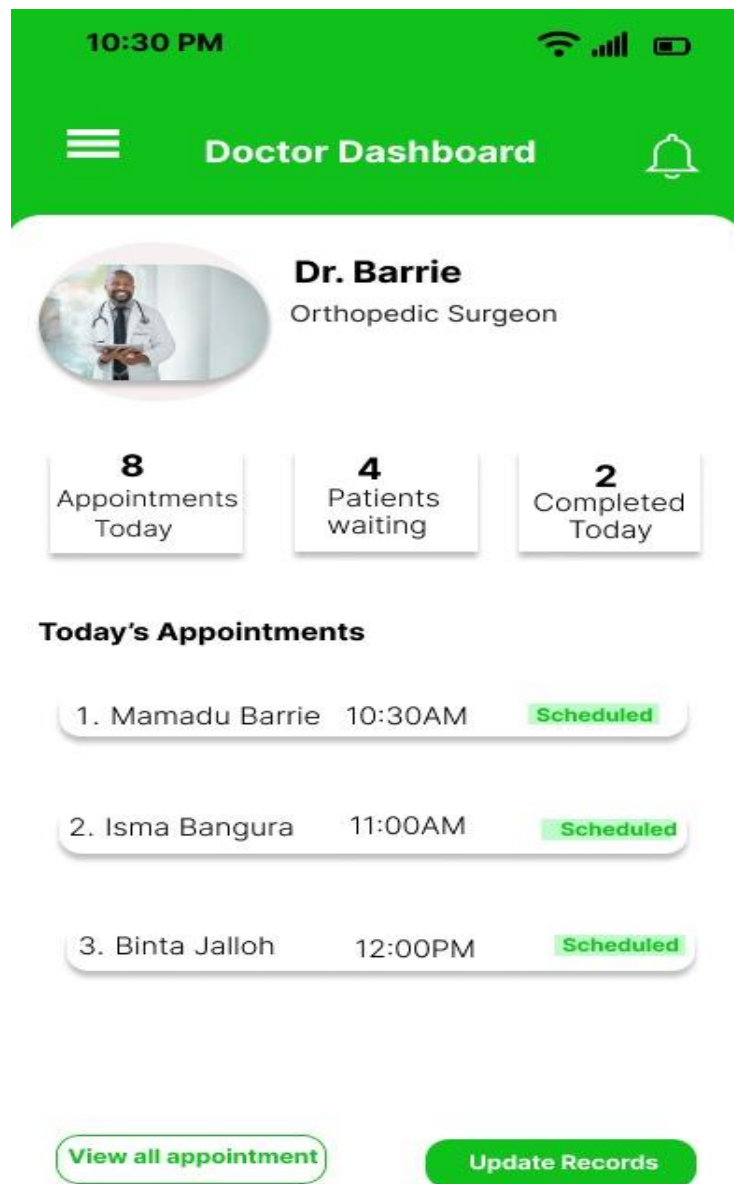


Figure 12: Doctor Dashboard

5.5 Administrator Experience

Screen 14: Admin Dashboard

The administrator's dashboard summarises total users, total doctors, total appointments, and today's view count, with direct access to the three management functions defined in the use case diagram: Manage User, Manage Doctors, and Generate Reports, plus general System Settings.

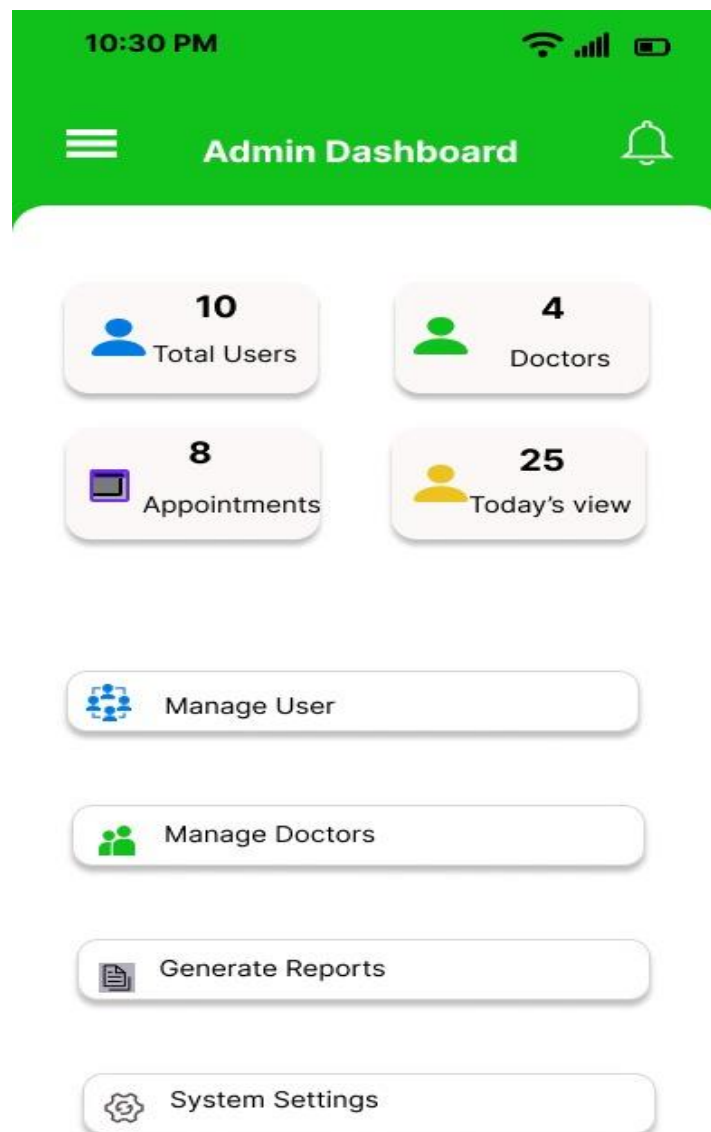


Figure 13: Admin Dashboard

5.6 Mapping Screens to Use Cases

Every screen in this section maps directly back to a use case from Figure 2, which is intentional: a Digital Public Good prototype should demonstrate that each documented requirement actually has a working interface behind it, not just a diagram.

Use Case (Figure 2)	Corresponding Screen(s)
Register / Login	Create Account, Login
Book Appointment	Find a Doctor, Book Appointment
View Medical Records	Medical Records
Receive Notification	Notifications
Manage Profile	Patient Home (Profile quick action)
View / Schedule Appointment	Doctor Dashboard
Update Medical Records	Doctor Dashboard (Update Records action)
View Patient History	Doctor Dashboard
Manage User / Manage Doctors / Generate Reports	Admin Dashboard

Table 2: Traceability between use cases and prototype screens

6. Git & GitHub Integration

Version control was managed through a dedicated GitHub repository created for the Smart Health Access System, used to store and track all project artefacts and to support collaboration between team members across the project's seven-week build.

6.1 Repository Structure

The repository is organised so that anyone reviewing the project, an examiner, a future contributor, or another student building on this work, can find any artefact without needing to ask where it is kept.

Folder / File	Contents
Software Documentation/ Project	Project report (this document), SDLC explanation, license file
Software/diagrams	Use case, class, sequence, architecture, Gantt, and network diagrams
UI or UX /prototype	Exported Figma screens and prototype link
README.md	Project overview, setup instructions, and SDG alignment statement
LICENSE	MIT License text

Table : GitHub repository structure

6.2 Collaboration Workflow

Work was committed incrementally in line with the Spiral SDLC model described in Section 2: each completed screen, diagram, or document section was committed with a descriptive message rather than committing the entire project at once, which keeps the commit history itself a useful record of how the project evolved. Branches were used to separate design work from documentation work so that one did not block the other, and changes were merged back into the main branch once reviewed.

- main branch: holds the stable, reviewed version of all project artefacts

- feature/prototype-screens branch: used while building and refining the Figma exports
- feature/documentation branch: used while drafting and revising this report
- Commits reference the relevant project task (e.g. "Add Book Appointment screen — Task E") for traceability against the Gantt chart in Section 7

6.3 README and Onboarding

The repository's README.md gives a short project description, the SDG 3 alignment statement, setup instructions for running the client and server locally, and a link to the live Figma prototype, so that the repository is usable by someone encountering the project for the first time and not only by its original authors, which is part of what makes a GitHub repository genuinely support a Digital Public Good rather than merely store files.

7. Project Plan

The project was planned across the seven weeks allotted for the Final Project (Weeks 5 to 11), with each week assigned to one or more of nine tasks. A Gantt chart shows the planned duration and timing of each task, and a network diagram shows the dependencies between tasks and identifies the critical path, the sequence of tasks that determines the minimum possible length of the entire project.

7.1 Gantt Chart

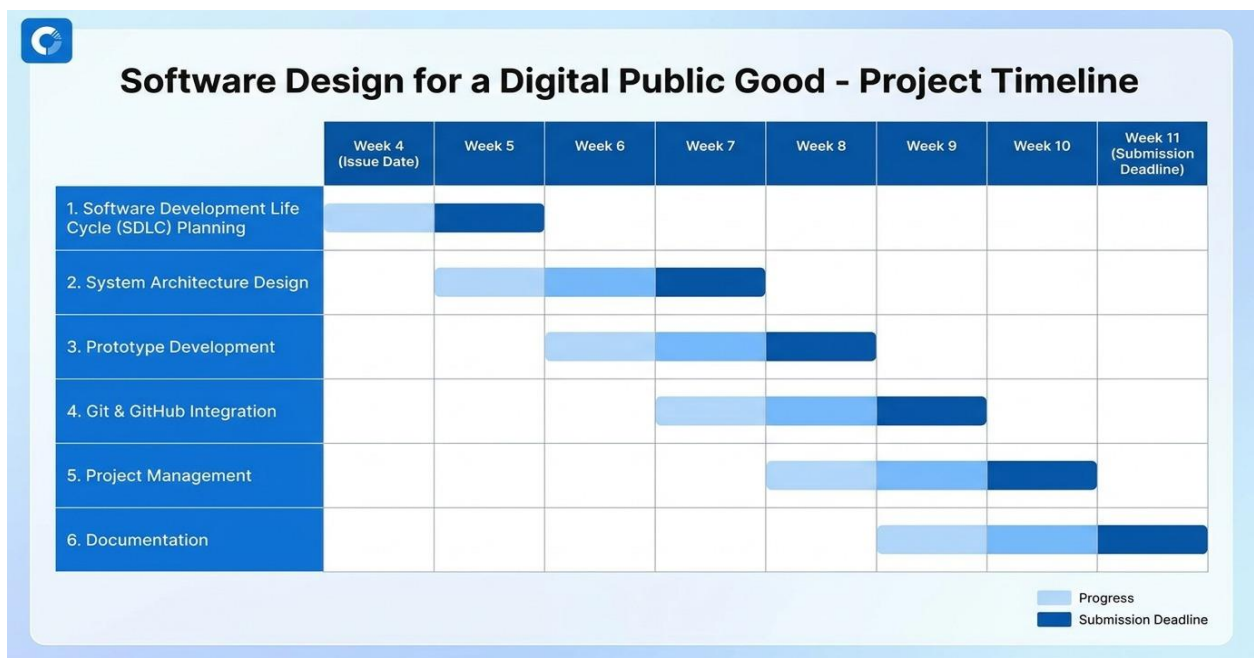


Figure 14: Project Gantt Chart, Week –11

Two tasks, GitHub Setup & System Architecture (Task C) and UML Diagrams (Task D), were scheduled in parallel during Week 6, since architecture decisions and the class/sequence diagrams informed each other and did not need to wait on one another to begin. Every other task proceeds sequentially: each depends on the deliverable from the task before it, for example the UI/UX Design in Week 7 depends on the system architecture from Week 6 being finalised first.

7.2 Network Diagram & Critical Path

Software Engineering Project Network Diagram (PERT/CPM)

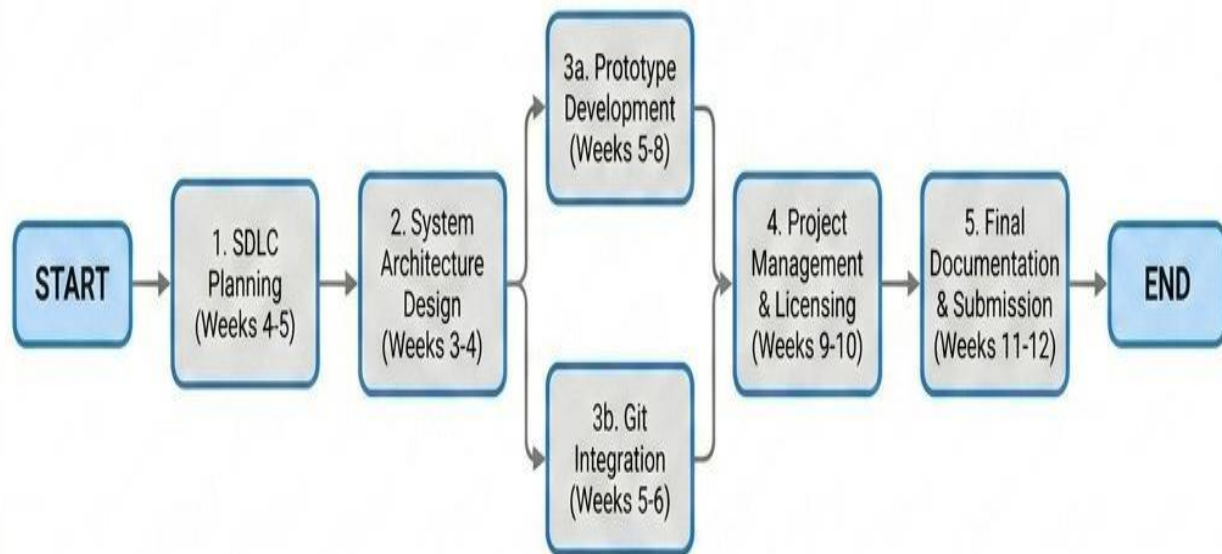


Figure 15: Network Diagram showing task dependencies and the critical path

The critical path, highlighted with a dark outline in Figure 16, runs through Research (A), SDLC Planning (B), GitHub Setup & Architecture (C), UI/UX Design (E), Prototype Build (F), License & SDG Review (G), Documentation (H), and Final Review (I). This is the longest chain of dependent tasks in the project, so any delay along this path delays the whole project, whereas the UML Diagrams task (D) has slack: it could run a few days later without affecting the submission deadline, since its only downstream dependency is the UI/UX Design task it feeds into alongside Task C.

7.3 Task Summary

Task	Description	Duration
A	SDG & Requirements Research	2 days
B	SDLC Model & Iteration Planning	5 days
C	GitHub Setup & System Architecture	5 days
D	UML Diagrams	2 days
E	UI/UX Design — 10 Figma Screens	2 week
F	Functional Prototype Build	4 days
G	License Selection & SDG Review	2 days
H	Report Documentation	3 weeks
I	Final Review & Submission	1 day
	Total	8 Weeks

Table 5: Project task summary

8. License, Design Quality & SDG Relevance

8.1 License Selection

The Smart Health Access System is released under the MIT License. The MIT License was chosen because it is short, widely understood, and places almost no restriction on how others may use, modify, or distribute the software, requiring only that the original copyright notice and license text be preserved in any copy or substantial portion of the software. For a project whose explicit goal is to function as a Digital Public Good, freely reusable by other developers, clinics, or NGOs facing the same access-to-healthcare problem, a permissive license like MIT removes legal friction that could otherwise prevent adoption, which a more restrictive or copyleft license could introduce.

The full license text is stored in the repository's LICENSE file at the root of the GitHub repository described in Section 6, and a short licensing notice is included in the README so that anyone discovering the project understands their usage rights immediately, without needing to open a separate file first.

8.2 Attribution of External Resources

Several third-party resources were used in building the prototype and are credited here in line with their own usage terms. The Figma design tool was used to design and prototype all ten screens shown in Section 5. Illustration assets used on the Welcome and Get Started screens are stock vector illustrations licensed for design use. Icons used throughout the interface follow common open icon-set conventions (calendar, bell, profile, and similar symbols) rather than any single proprietary icon library. No copyrighted text, branding, or imagery belonging to an existing commercial health platform was reproduced; all screen content, copy, and sample data (such as the doctor names and sample medical record entries shown in Section 5) were created specifically for this project.

8.3 Design Quality (UI/UX)

Design quality was treated as a first-class requirement rather than an afterthought, consistent with the marking criteria in the project brief. A single colour palette, primarily green to evoke health and trust, paired with white space and rounded cards, is applied consistently across all ten screens. Navigation follows a small number of clear patterns: a fixed bottom tab bar for the patient's four core sections (Home, Records, Alerts, Profile), a consistent header with a back arrow and notification bell on every secondary screen, and large, clearly labelled primary action buttons (Login, Register, Confirm Booking) that are never ambiguous about what they will do.

Information hierarchy was also considered deliberately: each dashboard leads with the two or three numbers a user most needs (a patient's upcoming appointments and new notifications; a doctor's appointments today, patients waiting, and completions; an admin's total users, doctors, appointments, and today's views) before offering secondary actions, rather than presenting every possible option with equal visual weight.

8.4 SDG Relevance & Real-World Impact

The system's alignment with SDG 3 is not a label applied after the fact; it shapes specific design decisions. Medication and appointment reminders (Figure 12) directly support treatment adherence, a known barrier to good health outcomes when patients are responsible for remembering multi-day medication schedules without support. The downloadable PDF medical record (Figure 11) gives patients a portable copy of their own data, useful in settings where a patient may need to see a different doctor or clinic than the one that originally treated them. The doctor's patient-history view (Section 5.4) ensures continuity of care even when a patient is seen by a doctor who has not treated them before, which matters in clinics with multiple practitioners or high staff turnover.

Taken together, these features describe a realistic, deployable improvement to how a small clinic or a network of community health workers could operate today, rather than a speculative or purely academic exercise, which is the real-world problem-solving value the project brief asks the system to demonstrate.

9. Conclusion

This report has presented the complete design and prototype of the Smart Health Access System, a Digital Public Good aligned with Sustainable Development Goal 3. The project was developed using an Iterative and Incremental SDLC model, supported by a three-tier system architecture, modelled through use case, class, and sequence UML diagrams, planned through a Gantt chart and network diagram, and brought to life through a ten-screen, high-fidelity Figma prototype covering the Patient, Doctor, and Administrator roles. The project is version-controlled on GitHub and released under the MIT License to support its goal of being freely reusable by others working on the same real-world problem.

Beyond satisfying the project's assessment criteria, the Smart Health Access System demonstrates how a digital public good can be approached end-to-end: starting from a genuine access-to-healthcare problem, working through a defensible architecture and data model, and arriving at a prototype that a patient, doctor, or administrator could plausibly use today.

10. References

- Figma (2026) *Figma: The collaborative interface design tool*. Available at: <https://www.figma.com> (Accessed: 10 June 2026).
- GitHub (2026) *GitHub: Build and ship software on a single, collaborative platform*. Available at: <https://github.com> (Accessed: 10 June 2026).
- International Organization for Standardization (ISO) (2011) *ISO/IEC 25010: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE)*. Geneva: ISO.
- Object Management Group (OMG) (2017) *Unified Modeling Language (UML), Version 2.5.1*. Needham, MA: OMG.
- The Linux Foundation (2026) *The MIT License (MIT)*. Available at: <https://opensource.org/licenses/MIT> (Organizatio0 June 2026).
- United Nations (2026) *Goal 3: Ensure healthy lives and promote well-being for all at all ages*. Available at: <https://sdgs.un.org/goals/goal3> (Accessed: 10 June 2026).
- Sommerville, I. (2016) *Software Engineering*. 10th ed. Boston: Pearson.